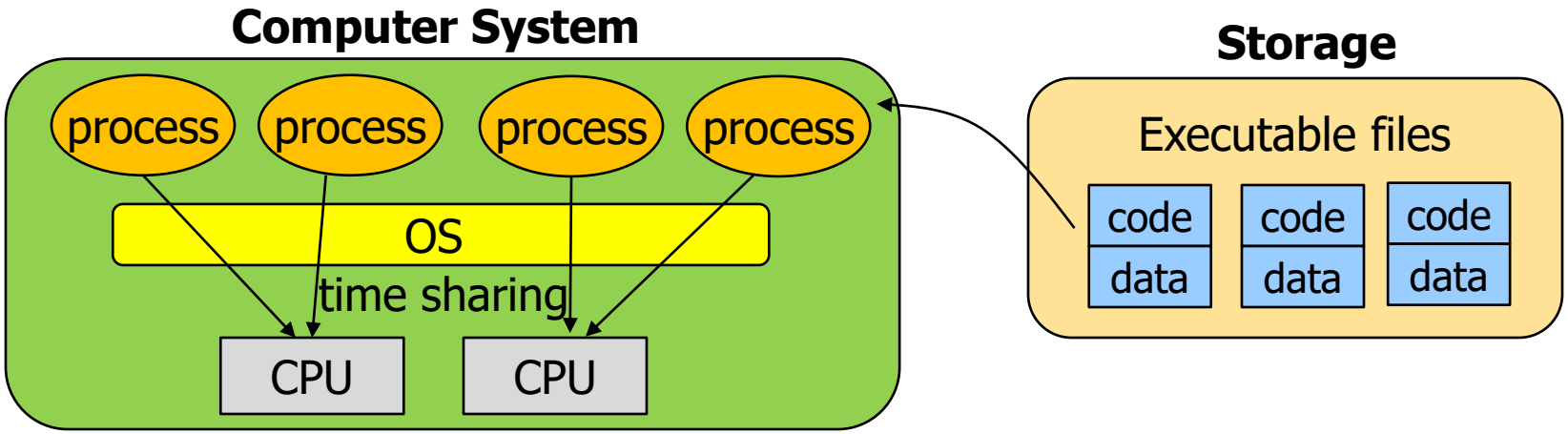
Chap 4, 5: Process

Process Concept

- Process (= running program)
  - An entity that is registered to kernel for execution
  - Kernel manages the processes to improve overall system performance
- A typical system may be seemingly running tens or even hundreds of processes at the same time.
- CPU Virtualization
  - There are only a few physical CPUs available,
  - The OS can promote the illusion that many virtual CPUs exist via **time sharing**
  - By running one process, then stopping it and running another.



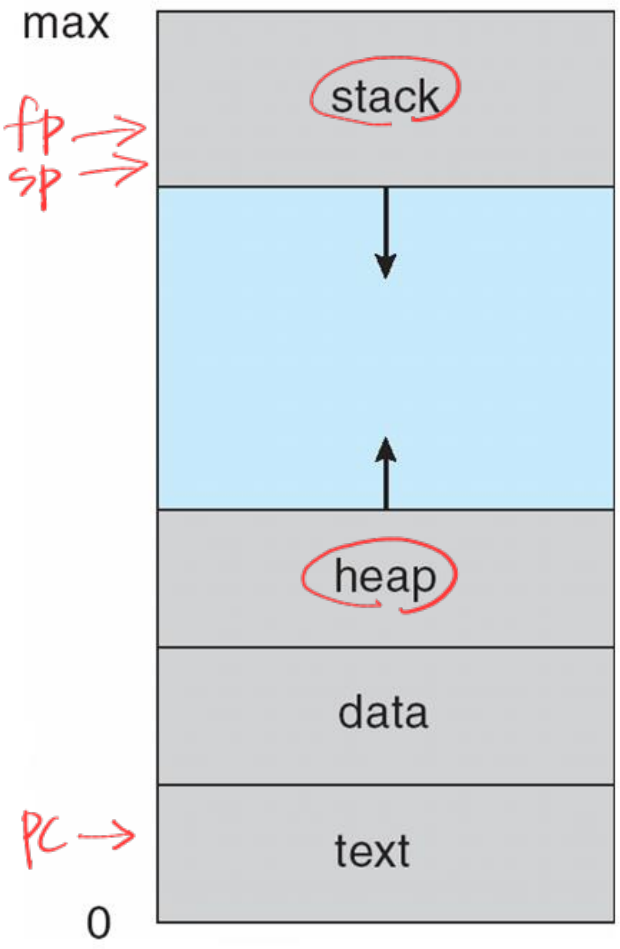
Process Concept

- A process is
  - A program in execution
  - An entity that is registered and being managed by kernel
  - An active entity
    - Request/allocate/release system resources during execution
  - An entity that is allocated the PCB

- **PCB** (Process Control Block)
  - Keeps several information about each process that is registered to the kernel
  - Maintained in kernel space

Machine State of Process

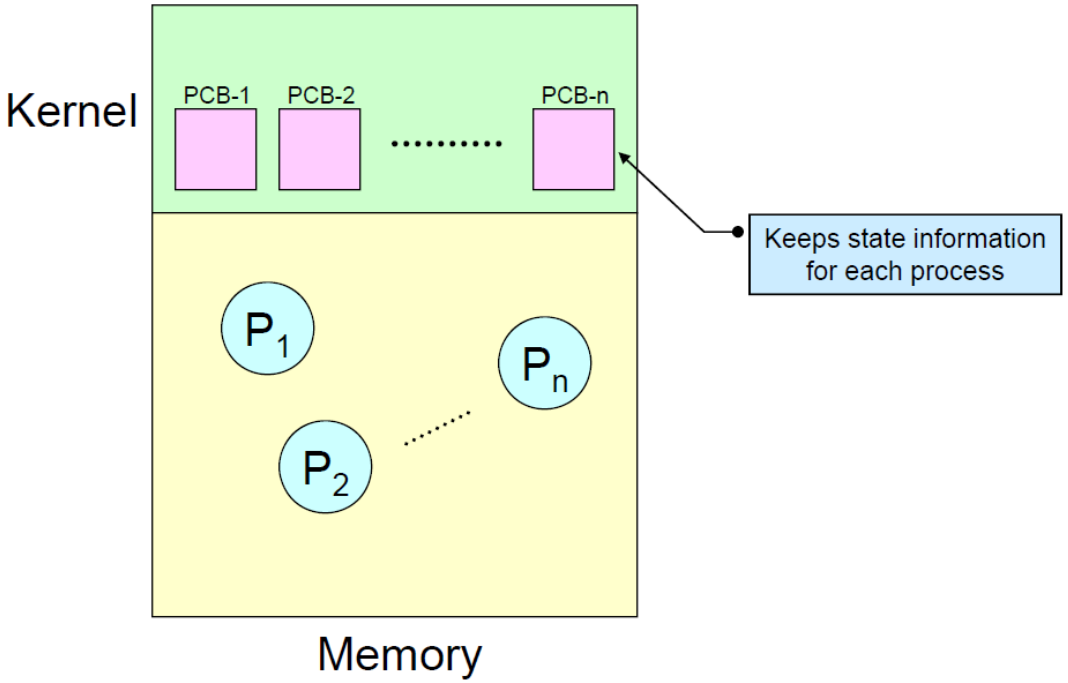
- A process includes
  - Memory
    - Program code → text
    - Global data → data
    - Temporary data → stack
      - Local variables, function parameters, return addresses
    - Heap
      - Memory area that is dynamically allocated during execution
  - Registers
    - Values of the processor registers
    - Include program counter, stack pointer, frame pointer
  - I/O information
    - a list of the files the process currently has open



Process memory address space

PCB

Information an OS needs to track about each process



PCB

- **Information in PCB (task control block)**
  - PID (Process Identification Number)
  - Process state: running, waiting, etc
  - Program counter: location of instruction to next execute
  - CPU registers: contents of all process-centric registers
  - Scheduling information
    - Process priority, scheduling parameters, etc.
  - Memory management information
    - Base/limit registers, page tables, segment tables, etc.
  - I/O status information
    - List of I/O devices allocated, list of open files, etc.
  - Accounting information
    - CPU used, clock time elapsed since start, time limits
  - Context save area
    - Space for saving register context of the process

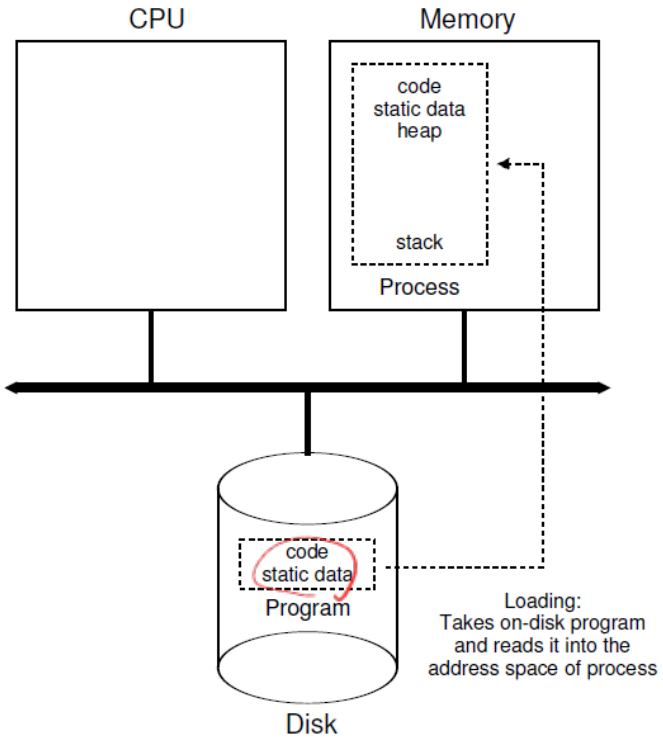
Process API

- **Create**
  - Some method to create new processes.
  - OS is invoked to create a new process to run the program you have indicated.
- **Destroy**
  - an interface to destroy processes forcefully.
- **Wait**
  - Wait for a process to stop running
- **Miscellaneous Control**
  - suspend a process (stop it from running for a while) and then resume it (continue it running).
- **Status**
  - get some status information about a process
  - such as how long it has run for, or what state it is in.

대부분의 process는 exit를 call해서 exit할, 그렇지 않은 경우는 kill해주어야 함

Process Creation

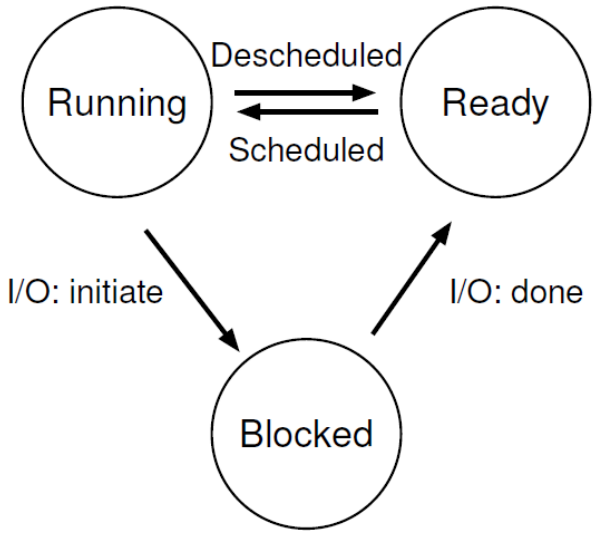
1. Programs initially reside on **disk** in some kind of **executable format**
2. **load** a program's code and any static data into memory, into the address space of the process.
   - Eager or lazy loading (paging and swapping)
3. Allocate run-time stack & initialize it with arguments (argc, argv)
4. Allocate heap
5. Do other initialization tasks related to I/O setup
6. jump to the main() routine, transfers control of the CPU to the newly-created process



Process States

- **Running**
  - a process is running on a processor, executing instructions.
- **Ready**
  - a process is ready to run but for some reason the OS has chosen not to run it at this given moment.
- **Blocked**
  - a process has performed some kind of operation that makes it not ready to run until some other event takes place.
  - A common example: when a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.

Process State Transition



Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	$Process_0$ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	$Process_1$ now done

Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	$Process_0$ initiates I/O
4	Blocked	Running	$Process_0$ is blocked,
5	Blocked	Running	so $Process_1$ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	$Process_1$ now done
9	Running	–	
10	Running	–	$Process_0$ now done

Process Creation API – fork()

- UNIX examples
- fork() system call creates new process
  - Child duplicate of parent address space, exact copy of the calling process

share same memory space

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) { // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else { // parent goes down this path (main)
        printf("hello, I am parent of %d (pid:%d)\n", rc, (int) getpid());
    }
    return 0;
}
```

Return value of fork()
- Parent: child PID
- Child: 0

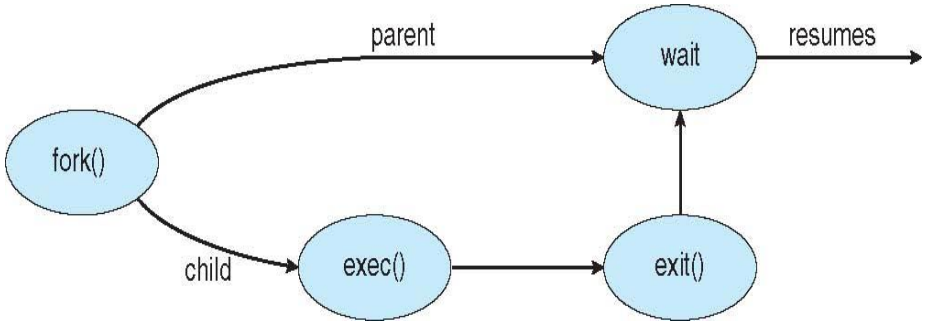
```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

or

```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

Process Creation API – wait()

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <unistd.h>
4   #include <sys/wait.h>
5
6   int
7   main(int argc, char *argv[])
8   {
9       printf("hello world (pid:%d)\n", (int) getpid());
10      int rc = fork();
11      if (rc < 0) {          // fork failed; exit
12          fprintf(stderr, "fork failed\n");
13          exit(1);
14      } else if (rc == 0) { // child (new process)
15          printf("hello, I am child (pid:%d)\n", (int) getpid());
16      } else {               // parent goes down this path (main)
17          int wc = wait(NULL);
18          printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
19                 rc, wc, (int) getpid());
20      }
21      return 0;
22  }
```



The return value of wait() is the child PID.

```
prompt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (wc:29267) (pid:29266)
prompt>
```

Wait 사용!

The child will always print first.

Process Creation API – exec()

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int
8  main(int argc, char *argv[])
9  {
10     printf("hello world (pid:%d)\n", (int) getpid());
11     int rc = fork();
12     if (rc < 0) {          // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child (new process)
16         printf("hello, I am child (pid:%d)\n", (int) getpid());
17         char *myargs[3];
18         myargs[0] = strdup("wc");   // program: "wc" (word count)
19         myargs[1] = strdup("p3.c"); // argument: file to count
20         myargs[2] = NULL;           // marks end of array
21         execvp(myargs[0], myargs);  // runs word count
22         printf("this shouldn't print out");
23     } else {                  // parent goes down this path (main)
24         int wc = wait(NULL);
25         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
26                rc, wc, (int) getpid());
27     }
28     return 0;
29 }
```

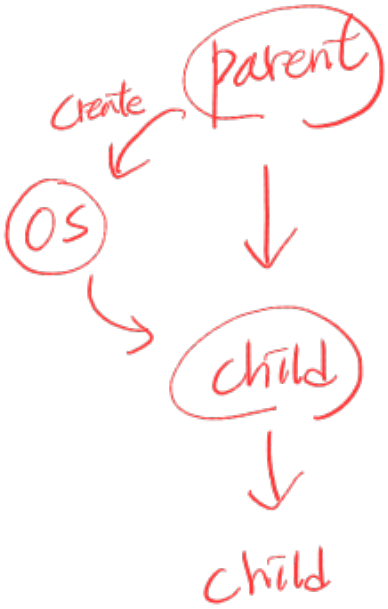
- exec() system call used after a fork() to replace the process' memory space with a new program
- On Linux, there are six variants of exec()
  - execl, execlp(), execle(), execv(), execvp(), execvpe()

a successful call to exec() never returns

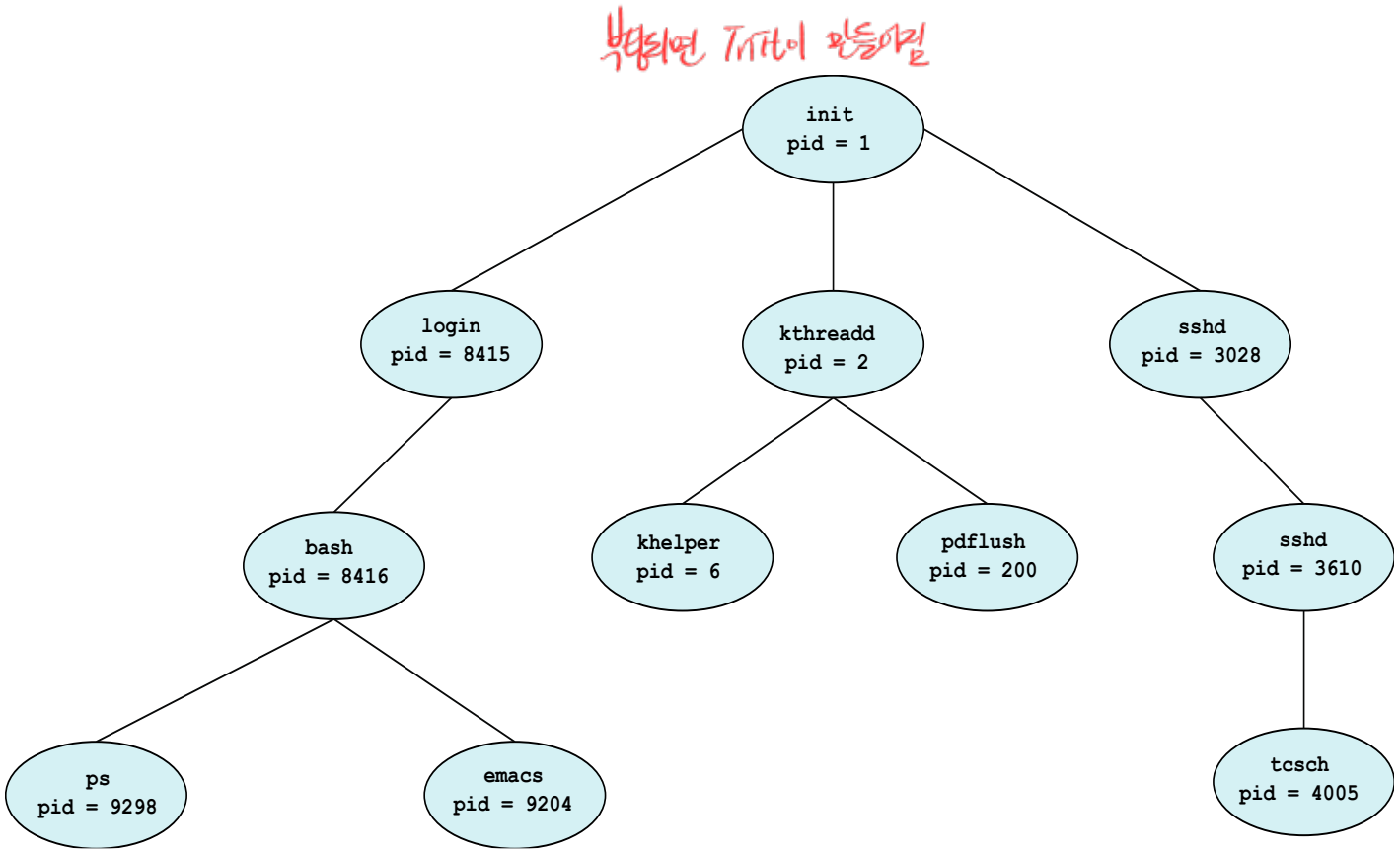
```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
      29     107    1030 p3.c
hello, I am parent of 29384 (wc:29384) (pid:29383)
prompt>
```

Process Creation

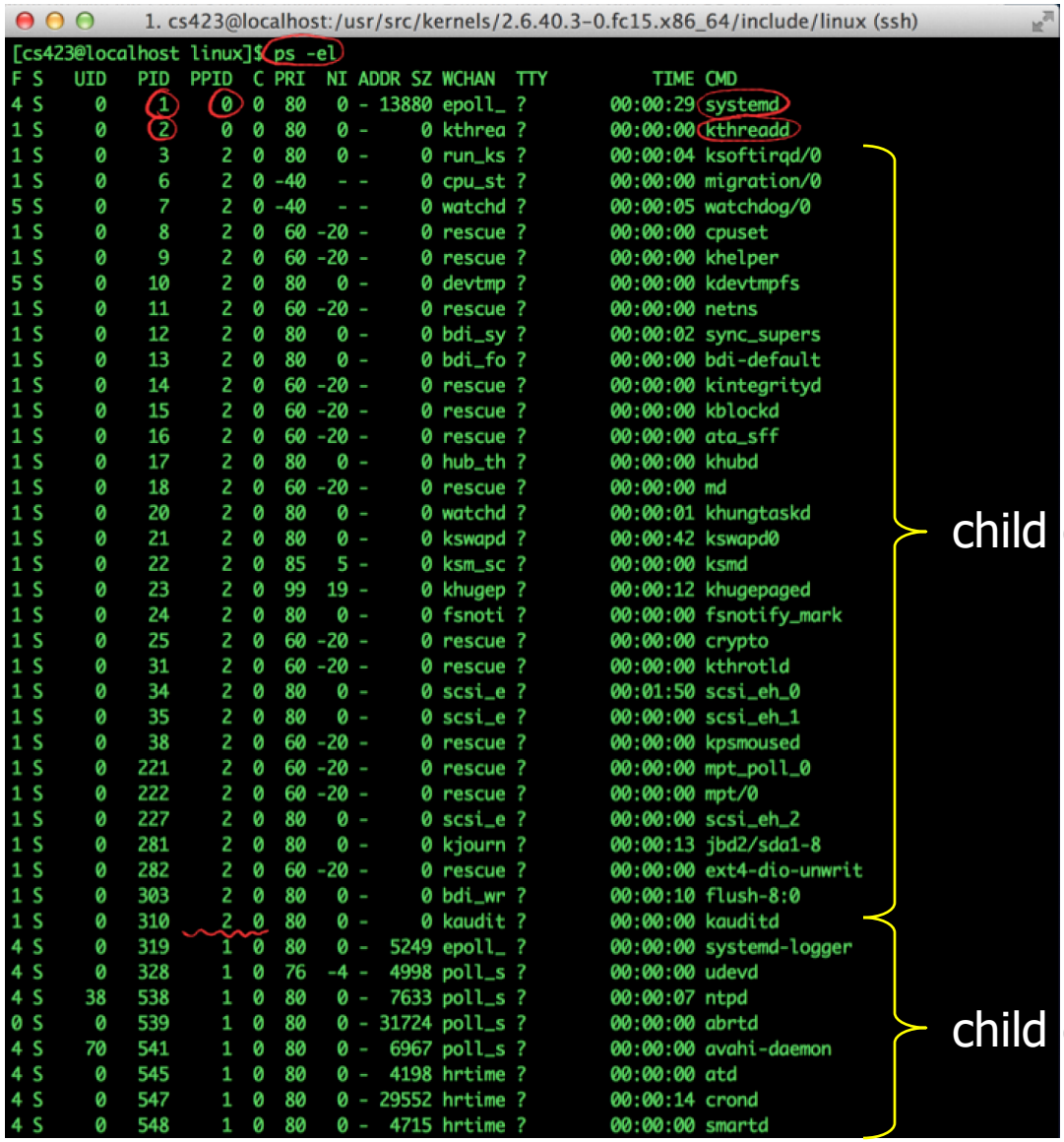
- Parent process create children processes, which, in turn create other processes, forming a tree of processes
- Generally, process identified and managed via a process identifier (pid)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate



Process Tree in Linux



'PS' command in Linux



systemd = init

child of kthreadd (kernel thread)

child of systemd (user thread)

Process Termination

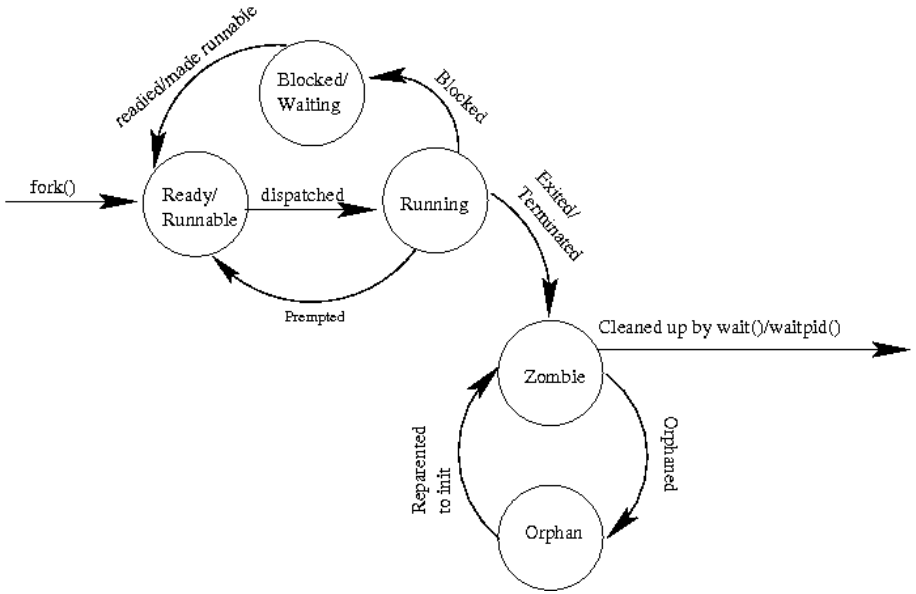
- **Process executes last statement and then asks the operating system to delete it using the `exit()` system call.**
  - Returns status data from child to parent (via **`wait()`**)
  - The resources of the now-extinct process are deallocated by operating system
- **Parent may terminate the execution of children processes using the `abort()` system call. Some reasons for doing so:**
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - Some operating systems does not allow a child to continue if its parent terminates.
    - All its children must also be terminated.
    - Cascading termination (All children, grandchildren, etc)
    - The termination is initiated by the operating system.

Zombie & Orphan

- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

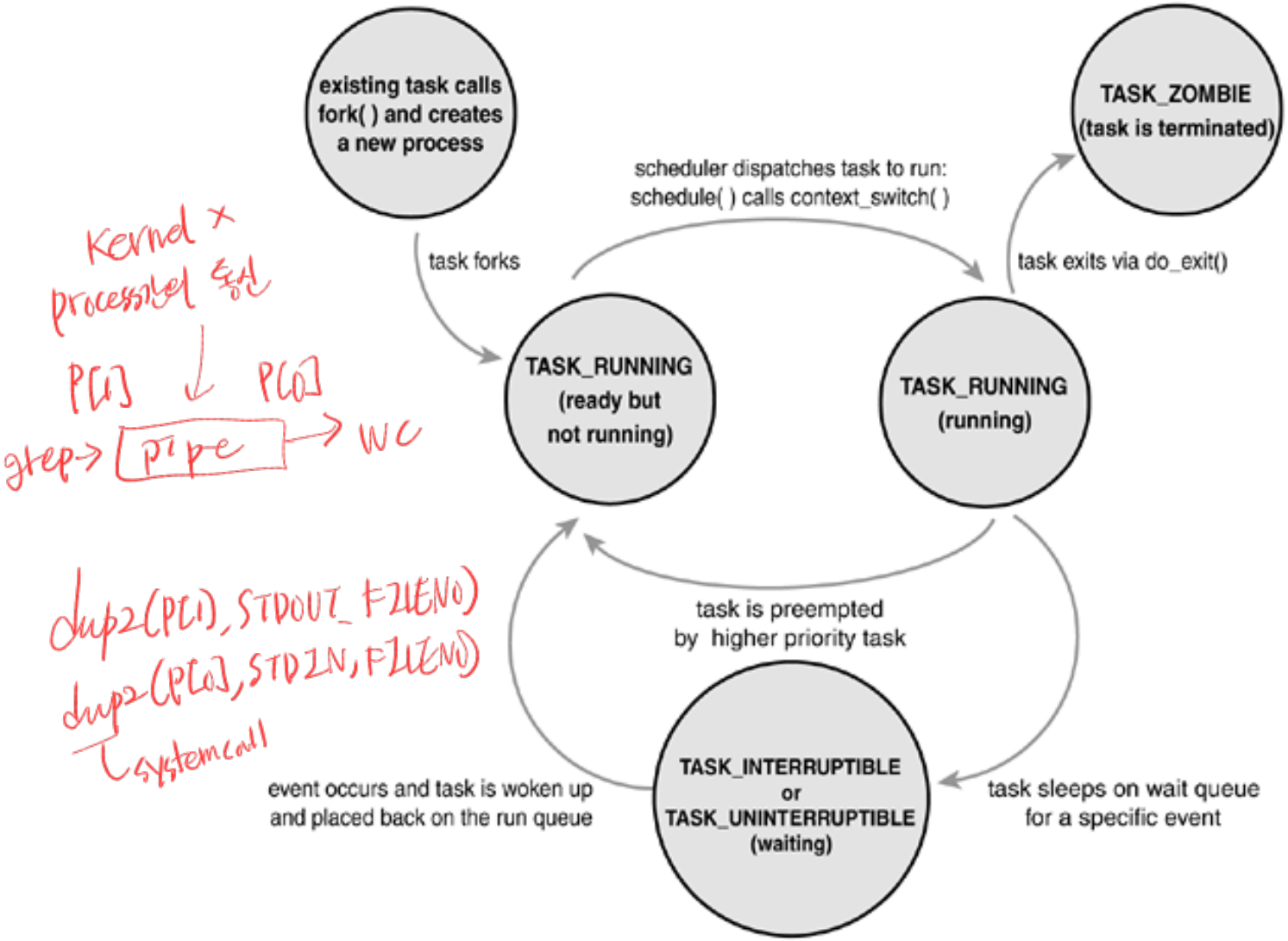
```
pid = wait(&status);
```

- If no parent waiting (did not invoke `wait()`) process is a zombie
  - It has exited but has not yet been cleaned up
- If parent terminated without invoking `wait`, process is an orphan
  - May be reparented and cleaned up by init.
  - In modern Linux systems, an orphan process may be reparented to a "subreaper" process instead of init. (see prctl(2))



reaper

Process States in Linux



Process Control and Users

- **Process control is available in the form of signals, which can cause jobs to stop, continue, or even terminate.**
- `int kill(pid_t pid, int sig)`
  - Send signals to a process
- **certain keystroke combinations are configured to deliver a specific signal to the currently running process**
  - control-c: SIGINT (interrupt) to terminate the process
  - control-z: SIGTSTOP (stop) to pause the process
- `sighandler_t signal(int signum, sighandler_t handler)`
  - Catch signals
  - SIGKILL and SIGSTOP cannot be caught or ignored.
- **Who can send a signal to a process?**
  - OS allows multiple users onto the system and ensures users can only control their own processes.
  - A superuser can control all processes.

Quiz

1. When a process is blocked, its PCB is deallocated and removed from memory. (X) process가 완료되어 종료될 때 PCB가 deallocate됨
2. PCB is updated only when a process is created or terminated. (X) PCB는 process state의 변화가 생길때 update됨
3. The parent and child share the same memory space and the same process ID. (X) 독립된 memory, 다른 pid
4. If wait() is called before a child process terminates, the parent process blocks until the child terminates. (O)
5. After a fork() call, the child process receive its process ID as a return value. (X) 0
6. The child process created by fork() starts executing from the beginning of the parent's code. (X) fork() 다음부터 execute
7. The execv() system call replaces the current process with a new program. (O)

xv6 Proc Structure

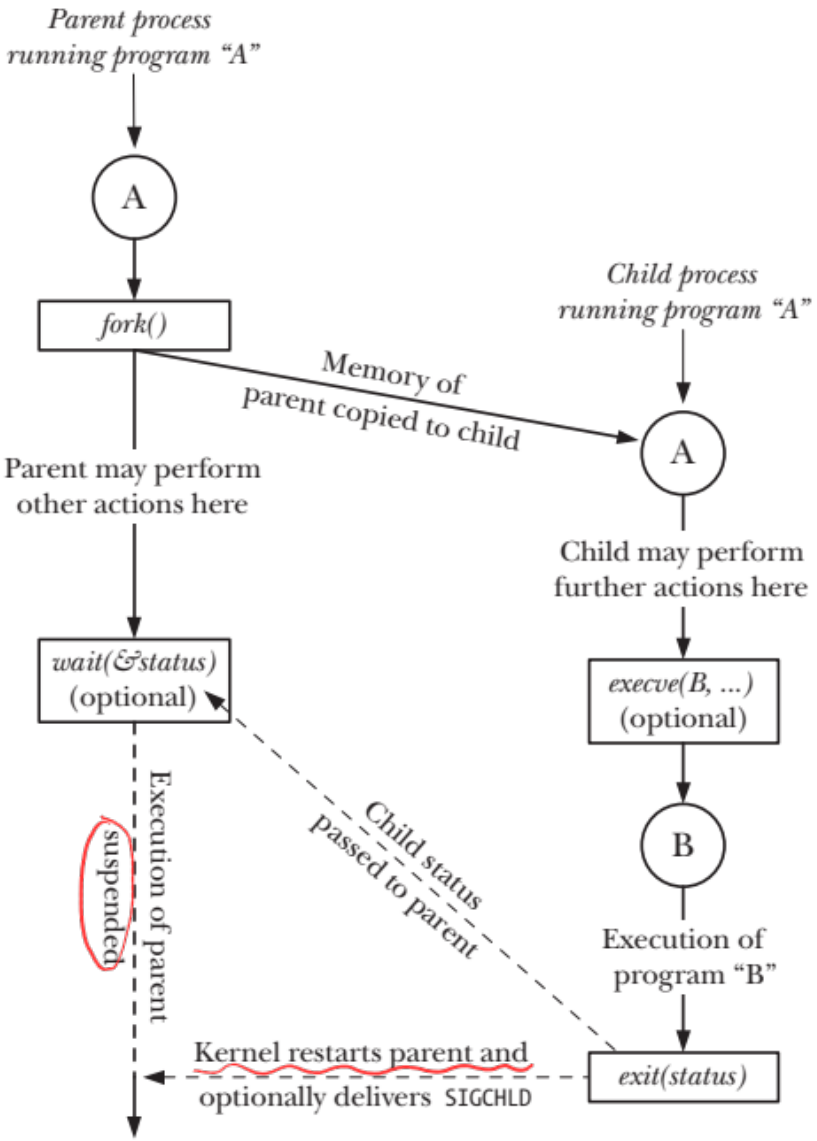
```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
  int eip;
  int esp;
  int ebx;
  int ecx;
  int edx;
  int esi;
  int edi;
  int ebp;
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

// the information xv6 tracks about each process
// including its register context and state
struct proc {
  char *mem;                  // Start of process memory
  uint sz;                    // Size of process memory
  char *kstack;               // Bottom of kernel stack
                              // for this process
  enum proc_state state;      // Process state
  int pid;                    // Process ID
  struct proc *parent;        // Parent process
  void *chan;                 // If !zero, sleeping on chan
  int killed;                 // If !zero, has been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;          // Current directory
  struct context context;     // Switch here to run process
  struct trapframe *tf;       // Trap frame for the
                              // current interrupt
};
```

For a stopped process, hold the contents of its registers.

Process Creation API – wait()



The separation of fork() and exec()

- **The shell calls fork() to create a new child process to run the command, calls exec() to run the command, and then waits for the command to complete by calling wait().**
  - can run some code between fork() and exec() to alter the environment of the about-to-be-run program.

```
prompt> wc p3.c > newfile.txt
```

The output of the program wc is redirected into newfile.txt.
Before calling exec(), the shell closes standard output and opens the file newfile.txt.
Any output from wc are sent to the file instead of the screen.

STDOUT_FILENO will be the first available one and thus get assigned when open() is called

```
prompt> ./p4
prompt> cat p4.output
      32     109     846 p4.c
prompt>
```

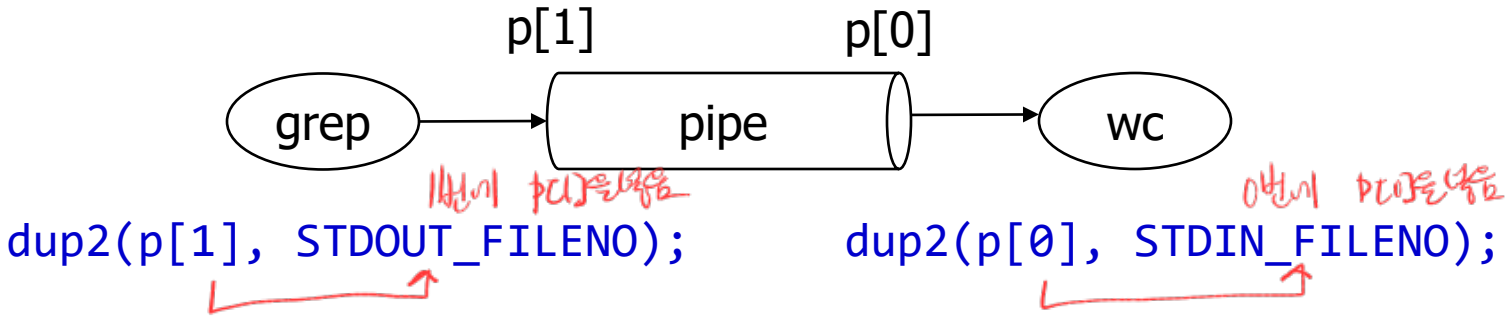
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
  int rc = fork();
  if (rc < 0) {
    // fork failed
    fprintf(stderr, "fork failed\n");
    exit(1);
  } else if (rc == 0) {
    // child: redirect standard output to a file
    close(STDOUT_FILENO);
    open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);

    // now exec "wc"...
    char *myargs[3];
    myargs[0] = strdup("wc");   // program: wc (word count)
    myargs[1] = strdup("p4.c"); // arg: file to count
    myargs[2] = NULL;           // mark end of array
    execvp(myargs[0], myargs);  // runs word count
  } else {
    // parent goes down this path (main)
    int rc_wait = wait(NULL);
  }
  return 0;
}
```

Pipe(): input/output redirection

```
grep -o foo file | wc -l
```



In Linux, STDOUT_FILENO and STDIN_FILENO are file descriptor constants used to refer to standard output (stdout) and standard input (stdin), respectively.
They are defined in <unistd.h> as:

```
#define STDIN_FILENO  0  // Standard input (keyboard)
#define STDOUT_FILENO 1  // Standard output (terminal)
#define STDERR_FILENO 2  // Standard error (terminal)
```

These constants allow programs to refer to standard input and output streams when working with system calls like read(), write(), and dup2().

Ask to ChatGPT

- **What happens in the Linux kernel when fork() is called?**
- **How different wait() and waitpid()? compare wait() and waitpid() with some code.**
- **How different zombie and orphan processes?**